

Throughput, Latency & Batching.

Serving an LLM well means balancing two metrics that pull against each other. Latency is what one user feels — the wait for the first token (prefill, compute-bound) plus the drip of each token after (decode, bandwidth-bound). Throughput is total tokens per second across all users, and it rises with batch size. Bigger batches help throughput but lengthen each step, so the two trade off along a curve with a knee. Continuous batching admits and evicts sequences every step to keep the GPU saturated. We derive end-to-end latency, build a batch-vs-throughput table, and compute tail percentiles. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$\text{latency} \approx \text{TTFT} + (n_{\text{out}} - 1) \cdot \text{TPOT} \quad \text{throughput} = \frac{B}{\text{TPOT}}$$

Worked example. TTFT = 80 ms, TPOT = 10 ms, $n_{\text{out}} = 200$: end-to-end = $80 + 199 \cdot 10 = 2070$ ms. Batch $B = 32$ at TPOT = 10 ms: $32/0.010 = 3200$ tok/s. From a 20-sample latency set: p50 = 102.5, p95 = 252.5, p99 = 290.5 ms.

Topic 3 of 3 in this module · companion to the course page · v1.0

Contents

1	The claim: two metrics, one trade-off	2
2	A diagram: static vs continuous batching	3
3	The derivation: end-to-end latency and throughput	3
4	Continuous (in-flight) batching	4
5	Tail latency: p50, p95, p99	4
6	Properties / consequences that matter	5
7	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 The claim: two metrics, one trade-off

Generation has two phases. **Prefill** processes the whole prompt in parallel and produces the first token; its cost, *time-to-first-token* (TTFT), grows with prompt length and is compute-bound (a big matrix multiply). **Decode** then emits one token per step; the per-token cost, *time-per-output-token* (TPOT, a.k.a. inter-token latency), is bandwidth-bound — each step reads the entire weight set and KV-cache to make one token. End-to-end latency for one request is prefill plus the remaining decode steps:

$$\text{latency} \approx \text{TTFT} + (n_{\text{out}} - 1) \cdot \text{TPOT} \quad (1)$$

Throughput is total tokens emitted per unit time. With a batch of B sequences each emitting one token per step of duration TPOT:

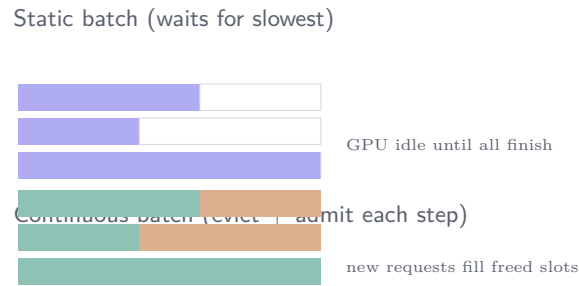
$$\text{throughput} = \frac{B}{\text{TPOT}} \quad (2)$$

The tension: raising B raises the numerator of Eq. (2) but, past the bandwidth knee, also raises TPOT (the denominator and a term in Eq. 1). Throughput climbs while per-request latency degrades.

Symbol	Meaning	Value / units
TTFT	time to first token (prefill)	80 ms
TPOT	time per output token (decode)	10 ms (at $B=32$)
n_{out}	output tokens generated	200
B	concurrent sequences in batch	$1 \rightarrow 128$
p50/p95/p99	latency percentiles	ms

Think of a bus. TTFT is how long until the bus departs once you board; TPOT is the time between stops. Running one nearly-empty bus gives each passenger a fast, smooth ride (low latency) but moves few people per hour (low throughput). Packing the bus moves many people per hour (high throughput) but each stop takes a little longer to load, so every passenger's trip lengthens. Continuous batching is letting people board and alight at every stop instead of waiting for a full new bus — the bus never runs empty and never waits for the slowest rider.

2 A diagram: static vs continuous batching



Static batching (top) holds every slot until the longest sequence finishes — short sequences leave the GPU idle (grey). Continuous batching (bottom) evicts a finished sequence (green) and immediately admits a waiting one (amber) at the next step, so the batch stays full and the GPU stays busy.

3 The derivation: end-to-end latency and throughput

End-to-end latency. With $\text{TTFT} = 80$ ms, $\text{TPOT} = 10$ ms, $n_{\text{out}} = 200$ tokens, the first token costs TTFT and the remaining 199 each cost TPOT:

$$\text{latency} = 80 + (200 - 1) \cdot 10 = 80 + 1990 = 2070 \text{ ms} = 2.07 \text{ s.}$$

Throughput. A batch of $B = 32$ sequences each emits one token per 10 ms step:

$$\text{throughput} = \frac{32}{10 \text{ ms}} = \frac{32}{0.010 \text{ s}} = 3200 \text{ tok/s.}$$

Batch dependence of TPOT. Decode reads a fixed cost (weights) every step plus a per-sequence cost (its KV-cache). A simple bandwidth model is $\text{TPOT}(B) = 8 + 0.0625 B$ ms: nearly flat while weight reads dominate, then linear once per-sequence reads add up. Substituting into Eq. (2) gives the throughput curve and its knee:

Batch B	TPOT (ms)	throughput (tok/s)	regime
1	8.06	124.0	latency-optimal, GPU starved
2	8.12	246.2	
4	8.25	484.8	
8	8.50	941.2	nearly free throughput
16	9.00	1777.8	
32	10.00	3200.0	approaching the knee
64	12.00	5333.3	balanced
128	16.00	8000.0	throughput-leaning, TPOT rising
			throughput-optimal, latency hurts

Below the knee ($B \lesssim 16$) TPOT barely moves, so throughput nearly doubles with each doubling of B for almost free. Above it ($B \gtrsim 64$) TPOT climbs fast: from $B=32$ to 128 throughput $2.5\times$'s but TPOT $1.6\times$'s, lengthening every request's latency by Eq. (1). The heatmap shades throughput (intensity $\propto$ tok/s, capped at indigo!80):

	$B=1$	4	16	32	64	128
tok/s	124	485	1778	3200	5333	8000

(Shading is linear in throughput relative to the largest, 8000 tok/s.)

Throughput and per-request latency trade off along the $\text{TPOT}(B)$ curve. Below the bandwidth knee you buy throughput almost for free (TPOT flat); above it every extra unit of throughput costs latency. Continuous batching keeps you near the knee by refilling the batch every step rather than draining it to the slowest sequence — worth roughly $2\text{--}3\times$ throughput over static batching.

4 Continuous (in-flight) batching

Static batching forms a fixed group and runs it to completion: a sequence that finishes at step 5 still occupies its slot until the longest sequence finishes at step 200, wasting 195 steps of that slot. **Continuous batching** re-forms the batch *every step*: completed sequences are evicted and waiting requests are admitted into the freed slots, so the batch stays near its capacity and the GPU stays saturated. Combined with PagedAttention (Topic 1), which makes admitting a new sequence a cheap block allocation, this lifts throughput roughly $2\text{--}3\times$ over static batching at the same hardware, because the average occupied batch size — not just the peak — is what Eq. (2) rewards.

5 Tail latency: p50, p95, p99

Averages hide the slow requests that violate SLAs. Take 20 sampled end-to-end latencies (ms):

{95, 97, 98, 98, 99, 99, 100, 101, 102, 102, 103, 104, 105, 106, 108, 109, 110, 210, 250, 300} (sorted).

The percentiles (linear interpolation, NumPy default) are

$$p50 = 102.5, \quad p95 = 252.5, \quad p99 = 290.5, \quad \text{mean} = 124.8 \text{ ms.}$$

The median request is fast (102.5 ms) and the mean (124.8 ms) looks fine, but 1 in 20 requests takes ≥ 210 ms and the p99 is 290.5 ms — nearly $3\times$ the median. An SLA written as “p99 ≤ 200 ms” is violated even though the average is well under it.

Statistic	Value (ms)	reads as
mean	124.8	misleadingly healthy
p50	102.5	typical request
p95	252.5	1 in 20 at least this slow
p99	290.5	1 in 100 at least this slow

6 Properties / consequences that matter

1. **Prefill vs decode have different bottlenecks.** TTFT is compute-bound (scales with prompt length); TPOT is bandwidth-bound (scales with weights + cache read per step). On the roofline, decode sits in the low-arithmetic-intensity, memory-bound region; prefill in the compute-bound region.
2. **Throughput is roughly linear in B below the knee.** Because TPOT is nearly flat there, B/TPOT grows almost proportionally to B .
3. **Latency degrades past the knee.** Once TPOT rises with B , Eq. (1) lengthens every request — the throughput gain is paid for in user-felt latency.
4. **Continuous batching maximizes average occupancy.** It is the average batch size over time, not the peak, that sets achieved throughput.

Optimizing the average while ignoring the tail violates SLAs. Here the mean (124.8 ms) and median (102.5 ms) both pass a “ ≤ 200 ms” target, yet p99 (290.5 ms) fails it. Tail latency is driven by queueing, long prompts (TTFT), and large batches (high TPOT) — exactly the levers you pull to raise throughput. Always quote and gate on p95/p99, not the mean; a system tuned purely for average latency or throughput will routinely breach a percentile SLA.

7 Where this sits in the track

This topic ties the module together. Topic 1 (PagedAttention) frees the cache memory that lets the batch B grow; Topic 2 (quantization) cuts the bytes moved per decode step, lowering TPOT at the bandwidth-bound knee. Module 3.1 (GQA) shrank the per-token cache read, also lowering TPOT. Continuous batching is the scheduler that turns all that freed capacity into sustained throughput while watching the latency percentiles that SLAs are written against.

A Reproducible (NumPy)

Inputs: $\text{TTFT} = 80$ ms, $\text{TPOT} = 10$ ms, $n_{\text{out}} = 200$; $\text{TPOT}(B) = 8 + 0.0625B$; the 20-sample latency set. The snippet reproduces the 2070 ms latency, the 3200 tok/s throughput, the batch table, and the percentiles (102.5, 252.5, 290.5).

```
import numpy as np
TTFT, TPOT, n_out = 80, 10, 200
print(TTFT + (n_out-1)*TPOT)          # 2070 ms
print(32/(TPOT/1000))                 # 3200.0 tok/s

for B in [1,2,4,8,16,32,64,128]:
    t = 8 + 0.0625*B                  # TPOT(B) ms
    print(B, round(t,2), round(B/(t/1000),1))
# 1 8.06 124.0 ... 32 10.0 3200.0 ... 128 16.0 8000.0

lat = np.array([95,102,98,110,105,99,210,108,101,97,
                103,250,100,106,99,104,98,109,102,300])
print(np.percentile(lat,[50,95,99]), lat.mean())
# [102.5 252.5 290.5] 124.8
```